CS/EE 147

GPU Computing & Programming

CUDA Midterm Study Guide

Prepared: April 21, 2026

Course Instructor: Nafis Mustakin
University of California, Riverside

TABLE OF CONTENTS

1. COURSE OVERVIEW & MOTIVATION

Why GPUs?

GPUs solve limitations of modern CPUs caused by Moore's Law slowdown, Dennard Scaling issues, and power constraints. Unlike CPUs designed for low-latency sequential processing, GPUs are optimized for **high-throughput parallel computing** with many simple cores.

GPU vs CPU Design

Aspect	CPU	GPU
Design Goal	Low latency	High throughput
Core Count	Few powerful	Many simple
Cache Size	Large	Small
Speed Advantage	10x+ on serial	10x+ on parallel

2. COMPUTER ARCHITECTURE FUNDAMENTALS

Memory Hierarchy

CPUs use multi-level caches (L1, L2, L3) to bridge gap between fast registers and slow DRAM. Virtual memory enables each process to have large virtual address space mapped to physical pages (typically 4KB).

Key Concepts:

• **Virtual Memory:** Pages mapped between virtual and physical memory

• **Process vs Thread:** Process = program instance; Thread = lightweight unit

• **ISA:** Instruction Set Architecture defines instruction formats

3. CUDA C PROGRAMMING BASICS

CUDA Programming Model

CUDA extends C with GPU-specific constructs. Programs have **host code** (CPU) and **device code** (GPU). NVCC compiler handles both using PTX (Parallel Thread eXecution) as intermediate virtual ISA.

Function Qualifiers

Qualifier	Executes On	Callable From	Purpose
__global__	Device (GPU)	Host (CPU)	Kernel function
__device__	Device	Device	Helper functions
__host__	Host	Host	CPU functions

Vector Addition Example

__global__ void vecAddKernel(float* A, float* B, float* C, int n) { int i = threadIdx.x + blockDim.x * blockIdx.x; if(i < n) C[i] = A[i] + B[i]; }
Each thread computes one element using thread indices.

4. CUDA PARALLELISM MODEL

Grid-Block-Thread Hierarchy

CUDA launches a **grid** of **blocks**, where each block contains **threads**. Threads within a block share shared memory and synchronize with __syncthreads(). Blocks are independent and can run in any order.

Thread Indexing

• threadIdx = index within block (x, y, z)

• blockIdx = index within grid (x, y, z)

• blockDim = dimensions of each block

• gridDim = dimensions of grid

5. SIMT EXECUTION MODEL

Warp Execution

A **warp** groups 32 consecutive threads executing in lockstep on SIMD hardware. Programmers see MIMD (independent threads), but hardware executes as SIMD groups.

Warp Divergence

When threads in a warp take different code paths, hardware **serializes** paths. All threads execute both branches with predicates determining which results are written. This kills parallelism and should be minimized.

PTX Virtual ISA

NVCC produces PTX bytecode with infinite registers. PTX is JIT-compiled by GPU driver, enabling binary compatibility across generations.

6. CUDA MEMORY HIERARCHY

Memory Types

Type	Scope	Lifetime	Speed	Declaration
Registers	Thread	Thread	Very Fast	Local vars
Shared	Block	Block	Fast	__shared__
Global	Grid/App	Application	Slow	__device__
Constant	Grid/App	Application	Medium	__constant__

Memory Coalescing

Arrange memory access so consecutive threads read consecutive memory locations. This combines individual accesses into fewer large transactions for high bandwidth.

Tiling/Blocking

Load data into shared memory, process tiles, then load new data. Reduces global memory bandwidth significantly.

7. PARALLEL PATTERNS & OPTIMIZATION

Reduction Pattern

Partition large data into chunks, process each in parallel, then use reduction tree to combine results. Used in MapReduce and distributed ML.

CUDA Streams

Enable asynchronous execution. Multiple kernels and memory operations can overlap. Use for pipelining computation and data transfer.

Pinned Memory

Pinned (page-locked) memory allocated with cudaHostAlloc() cannot be paged out. Required for DMA transfers. Faster than pageable memory.

8. DEBUGGING & PROFILING TOOLS

NVCC Compiler Flags

• -G = Device debugging symbols

• -lineinfo = Line information for profiling

• -Xcompiler = Forward flags to host compiler

Debugging Tools

• **cuda-memcheck:** Memory errors, races, uninitialized memory

• **cuda-gdb:** Device-aware debugger

• **Nsight Compute:** Kernel-level profiling

• **Nsight Systems:** System-level profiling

9. MIDTERM PREPARATION CHECKLIST

■ **GPU Architecture:** Warp execution, SIMT model, divergence, scheduling

■ **CUDA Programming:** Kernel syntax, thread/block dimensions, function qualifiers

■ **Memory:** Memory types, coalescing, shared memory, pinned memory

■ **Parallelism:** Grid/block/thread hierarchy, indexing, synchronization

■ **Optimization:** Memory bottlenecks, tiling, coalescing patterns

■ **Tools:** nvcc flags, cuda-memcheck, profilers

■ **Patterns:** Reduction, data-parallel, stencil computation

10. PRACTICE QUESTIONS

1. What is a warp and why is warp divergence problematic?

2. What are the constraints on threads per block?

3. Explain differences between shared memory and global memory.

4. Write a kernel for element-wise vector addition.

5. What is memory coalescing and why does it matter?

6. When do you use __device__, __global__, and __host__?

7. How would you optimize a kernel with poor memory utilization?

8. Explain PTX and virtual ISA advantages.

9. What is pinned memory and when should you use it?

10. Compare CPU and GPU design. When is each better?

11. How do you calculate 2D thread indices for image processing?

12. Explain the reduction tree pattern.

13. What causes warp divergence in conditional statements?

14. How does tiling improve kernel performance?

15. What profiler would you use to analyze kernel behavior?

EXAM TIPS:

- Review vector addition example and variations
- Practice thread index calculations
- Understand memory access patterns
- Know memory hierarchy and when to use each type
- Understand warp behavior and divergence